



Learn C++ by Building Idler

A hands-on course in modern C++, Qt, and cross-platform desktop development — taught through one real application.

From a blank folder to a signed, shippable app for Windows, macOS, and Linux.

For readers who have never written a line of C++.

Contents

Preface: How to Read This Book	3
Chapter 1 — What We Are Building, and What a Program Is	4
Chapter 2 — Setting Up Your Workshop	6
Chapter 3 — Your First Program, and How Code Becomes a Program	7
Chapter 4 — The Building Blocks: Types, Variables, Functions	9
Chapter 5 — Splitting Code: Headers and Declarations	11
Chapter 6 — Teaching the Computer to Build: CMake	13
Chapter 7 — Thinking in Objects: Classes	15
Chapter 8 — Pointers, References, and Who Owns the Memory	17
Chapter 9 — Building a GUI with Qt: Widgets, Layouts, the Event Loop	19
Chapter 10 — Making Things Happen: Signals and Slots	21
Chapter 11 — State and Time: The Timer and the Click Loop	23
Chapter 12 — One Codebase, Three Operating Systems	25
Chapter 13 — Shipping It: Building, Bundling, Distributing	28
Chapter 14 — Next Lesson: Build a Focus Timer	30

Preface: How to Read This Book

This is not a reference manual. It is a *guided build*. Every idea in these pages exists to help you understand one small, real program: **Idler**, a cross-platform auto-clicker with a graphical interface that runs on Windows, macOS, and Linux.

By the end you will have read, understood, and be able to rebuild every line of a program that has a real window, real buttons, talks to three different operating systems, and ships as a downloadable app. That is a genuine piece of software, not a toy that prints text to a black screen.

Who this is for

You have never written C++ before. Perhaps you have never written any code before. That is fine. We start from "what even *is* a compiler" and build up. Where a concept is deep enough to fill its own book (memory management, the C++ type system), we teach the slice you need to understand Idler, and we tell you honestly that there is more to learn later.

How each chapter works

- **We read real code.** Every snippet in this book is taken from the actual Idler source. Nothing is invented for the lesson.
- **We explain the "why", not just the "what".** Anyone can memorize syntax. The goal is that you understand *why* the code is shaped the way it is.
- **You are expected to build.** After you read a chapter, open the matching file in the repository and read it top to bottom. The code will feel familiar.

Note — Programming is learned in the hands, not the eyes. Read a chapter, then go type the code yourself. Break it. Fix it. That loop is the entire skill.

The tools you will meet

C++ (the language), CMake (the tool that builds the program), and Qt (the library that draws the window and buttons). Do not worry about what these words mean yet. Each gets its own chapter.

Let us begin.

Chapter 1 — What We Are Building, and What a Program Is

The app

Idler is an *auto-clicker*. You set how often it should click the mouse, choose when it should stop, and press **Start**. The computer then clicks the left mouse button for you, on a schedule, until you tell it to stop.

It has a small window with:

- a box to type the interval ("click every 2 seconds"),
- a choice of *mode*: click forever, click for a set amount of time, or click a set number of times,
- a **Start / Stop** button,
- a line of text showing what is happening ("Running — 14 clicks").

Small as it is, building this teaches almost everything a beginner needs: how code becomes a program, how to structure a project, how to draw a user interface, how to react to button presses, how to keep track of time, and how to speak to the operating system underneath.

What *is* a program?

A program is a file full of instructions that the computer's processor (the CPU) can execute. But the CPU only understands *machine code* — raw numbers. Humans cannot reasonably write machine code by hand, so instead we write **source code** in a language like C++ that is readable by people, and we use a tool called a **compiler** to translate it into machine code.

So the shape of our work is always:

- You write C++ source code in text files (ending in `.cpp` and `.h`).
- A compiler translates that into a machine-code **executable** (`idler.exe` on

Windows, `idler.app` on macOS, a plain `idler` file on Linux).

- The user runs the executable, and the program comes alive.

Why C++?

C++ is a *compiled, statically typed* language that produces fast, native programs with direct access to the operating system. For an app that needs to send real mouse clicks to the OS and draw a native window, C++ is a natural fit. It is also famously large and demanding — so we will be deliberate about learning only what we need, when we need it.

The map of the project

Here are the files that make up Idler. Do not try to understand them yet; this is the territory we will explore.

- `main.cpp` — the entry point; where the program starts.
- `MainWindow.h` / `MainWindow.cpp` — the window, its controls, and its behavior.

- `clicker.h` — a one-line promise: "there exists a function that clicks the mouse".
- `clicker_windows.cpp`, `clicker_mac.mm`, `clicker_linux.cpp` — three different implementations of that promise, one per operating system.
- `CMakeLists.txt` — the recipe that tells the build tool how to assemble everything.

Note — Notice already a central idea of good software: `clicker.h` states *what* is possible ("something can click the mouse"), while the three platform files decide *how*. Separating the "what" from the "how" is a theme we will return to often.

In the next chapter we set up the tools so you can build and run this yourself.

Chapter 2 — Setting Up Your Workshop

Before we can build, we need three things installed: a **C++ compiler**, the **CMake** build tool, and the **Qt** library. A code editor (VS Code is a fine free choice) rounds it out.

macOS

Install Apple's compiler by installing the Xcode Command Line Tools, then use Homebrew for the rest:

```
xcode-select --install
brew install cmake qt
```

Windows

Install **Visual Studio** (the Community edition is free) and, in its installer, tick "Desktop development with C++". That gives you the MSVC compiler and CMake. Then install Qt 6 from the official Qt online installer.

Linux (Debian / Ubuntu)

```
sudo apt install build-essential cmake qt6-base-dev libxtst-dev libx11-dev
```

`build-essential` brings the GNU C++ compiler (`g++`). The `libxtst-dev` package is needed because on Linux we click the mouse through the X11 "XTest" extension — more on that much later.

Checking it worked

Open a terminal and ask each tool its version:

```
cmake --version
```

If that prints a version number, CMake is on your PATH and ready. We will test the compiler by actually compiling something in the next chapter.

Note — A "terminal" or "shell" is the text window where you type commands. On macOS it is the Terminal app; on Windows, the "Developer Command Prompt for VS"; on Linux, your usual terminal. Throughout this book, lines you type into the terminal are shown in code boxes like the ones above.

Getting the code

Clone the repository so you can follow along in the real files:

```
git clone https://github.com/ikjbi/idler.git
cd idler
```

You now have every file we will study, sitting in a folder called `idler`.

Chapter 3 — Your First Program, and How Code Becomes a Program

Before touching Idler's real files, let us write the smallest possible C++ program and watch it turn into an executable. This demystifies the whole pipeline.

Hello, world

Create a file called `hello.cpp` with exactly this:

```
#include <iostream>

int main() {
    std::cout << "Hello, world!" << std::endl;
    return 0;
}
```

Compile and run it (on macOS or Linux):

```
c++ hello.cpp -o hello
./hello
```

You should see `Hello, world!` printed. You just wrote, compiled, and ran a C++ program. Let us dissect it, because every piece reappears in Idler.

Line by line

```
#include <iostream>
```

A line beginning with `#` is a **preprocessor directive** — an instruction handled *before* real compilation begins. `#include` literally pastes the contents of another file into yours. Here we include `iostream`, the standard library's input/output tools, which is what gives us `std::cout` (console output).

Idler's `main.cpp` opens the same way, including the tools it needs:

```
#include <QApplication>
#include <QIcon>
#include "MainWindow.h"
```

Angle brackets (`<...>`) mean "a library header the compiler knows where to find". Quotes (`"..."`) mean "a header file that lives in *my* project", like our own `MainWindow.h`.

```
int main()
```

Every C++ program has exactly one function named `main`. It is the **entry point**: when the program runs, execution begins at the first line inside `main` and the program ends when `main` finishes. The word `int` before it means `main` hands back an integer when it is done.

```
return 0;
```

That integer is the program's *exit code*. By convention `0` means "finished successfully"; any other number signals an error. The operating system reads this value.

Here is Idler's actual `main`, which you will fully understand by Chapter 9:

```
int main(int argc, char* argv[]) {
    QApplication app(argc, argv);
    app.setApplicationName("Auto Clicker");
    app.setWindowIcon(QIcon(":/assets/icons/icon_512.png"));

    MainWindow w;
    w.show();
    return app.exec();
}
```

Notice it still returns an integer, and it is still called `main`. The rest is just Idler-specific work sandwiched in between.

The three stages of building

When you ran `c++ hello.cpp -o hello`, three things happened in sequence:

- **Preprocessing** — all the `#include` and `#`-directives are resolved, producing one big stream of pure C++.
- **Compilation** — that C++ is translated into machine code, producing an *object file* for each source file.
- **Linking** — the object files, plus any libraries they use, are stitched together into the final executable.

Keep this three-step picture in mind. When we introduce CMake in Chapter 6, all it is really doing is running these steps for many files, in the right order, with the right settings.

Note — An error at *compile* time (e.g. a typo in your code) is very different from an error at *link* time (e.g. "you used a function but never provided its body"). Learning to tell them apart will save you hours. We will see a real linker error in Chapter 5.

Chapter 4 — The Building Blocks: Types, Variables, Functions

Now we learn the raw grammar of the language, using real fragments of Idler so the examples are never artificial.

Variables and types

A **variable** is a named box that holds a value. C++ is *statically typed*, which means every variable has a fixed **type** decided when you write the code, and the compiler enforces it. This catches a huge class of mistakes before the program ever runs.

The types you will meet in Idler:

- `int` — a whole number, e.g. `10`. Used for counting clicks.
- `double` — a number with a decimal point, e.g. `1.5`. Used for time intervals.
- `bool` — either `true` or `false`. Used for "are we currently running?".

Here are real member variables from Idler's window:

```
bool    m_running    = false;
int     m_clicksDone = 0;
double  m_elapsedSecs = 0.0;
```

Each line declares a variable, gives it a type, and sets a starting value. Read it as: "there is a boolean called `m_running`, and it starts out `false`." The `m_` prefix is a naming convention meaning "member" — we explain it in Chapter 7.

Note — Why types matter. Because `m_clicksDone` is an `int`, the compiler will stop you from accidentally storing the text "hello" in it. In a dynamically typed language that mistake might only surface when a user runs the program. C++ refuses to compile it. The strictness feels pedantic at first and becomes a safety net.

Constants

Sometimes a value must never change. Marking it `const` tells both the compiler and the next human reader "this is fixed". Idler uses a `const` table of conversion factors for turning seconds, minutes, and hours into a common unit:

```
static const double UNIT_FACTORS[] = { 1.0, 60.0, 3600.0 }; // s, min, hr
```

There are 60 seconds in a minute and 3600 in an hour; these numbers never change, so they are `const`.

Functions

A **function** is a named, reusable block of instructions. It can take **parameters** (inputs) and **return** a value (an output). We already met the most important function, `main`. Here is a tiny one from Idler:

```
double MainWindow::intervalSeconds() const {
    return m_intervalSpin->value() * UNIT_FACTORS[m_intervalUnit->currentIndex()];
}
```

Reading the first line as a sentence: "this function is called `intervalSeconds`, it returns a `double`, and it takes no parameters." The body computes a value and `returns` it. Ignore the `MainWindow::` prefix and the `const` for now; both are explained in Chapter 7.

The value of a function is that you write the logic *once* and call it from many places. Everywhere `Idler` needs to know "how many seconds is the chosen interval?", it simply calls `intervalSeconds()` instead of repeating the multiplication.

Arithmetic and expressions

An **expression** is anything that computes a value. The multiplication above is one. Here is another, converting a duration into milliseconds and guarding against a too-small value:

```
int intervalMs = qMax(50, static_cast<int>(intervalSeconds() * 1000.0));
```

Two new ideas here, both worth naming:

- `static_cast<int>(...)` **converts** a `double` into an `int` on purpose,

discarding the fractional part. C++ makes you ask for such conversions explicitly, because silently turning `1.9` into `1` is exactly the kind of surprise that hides bugs.

- `qMax(50, ...)` returns whichever argument is larger, so the interval can never

drop below 50 milliseconds. This is defensive programming: never trust that a value is sane, make it sane.

Control flow: making decisions

Programs choose between paths using `if`. `Idler` decides whether to stop after reaching a click target like this:

```
if (m_radioCount->isChecked() && m_clicksDone >= m_clickCountSpin->value()) {
    stop();
    return;
}
```

Read the condition as: "if the 'stop after N clicks' option is selected **and** the number of clicks done is greater-than-or-equal-to the target, *then* stop." The `&&` means "and"; both sides must be true. The `>=` is "greater than or equal to". If the condition holds, the code inside the braces runs.

With variables, functions, expressions, and `if`, you can already read a surprising amount of `Idler`. Next we learn how the code is split across files.

Chapter 5 — Splitting Code: Headers and Declarations

Real programs are not one giant file. Idler spreads its code across many `.cpp` files, and they need a way to know about each other. That is the job of **header files** (`.h`).

Declaration versus definition

This distinction is the key to the whole chapter.

- A **declaration** announces that something *exists* — its name and shape — without saying how it works.
- A **definition** provides the actual body — the real code.

Idler's `clicker.h` is a perfect, minimal example. In its entirety:

```
#pragma once

// Performs a left mouse click at the current cursor position.
void performLeftClick();
```

That is a *declaration*. It promises: "somewhere there exists a function called `performLeftClick` that takes no arguments and returns nothing (`void`)." It does **not** say how the click actually happens. Any file that says `#include "clicker.h"` can now *call* `performLeftClick()`, trusting that the body exists elsewhere.

Where is the body? In three separate files — one per operating system. Here is the macOS *definition* (simplified):

```
#include "clicker.h"
#include <ApplicationServices/ApplicationServices.h>

void performLeftClick() {
    // ... actually tell macOS to click ...
}
```

This split is powerful: `MainWindow.cpp` includes `clicker.h` and calls `performLeftClick()` without knowing or caring which operating system it is on. The right body gets connected at link time.

```
#pragma once
```

The first line of every Idler header is `#pragma once`. Because headers get `#included` into many files — and headers can include other headers — the same header might be pasted in twice, which would declare things twice and cause errors. `#pragma once` tells the compiler "include the contents of this file at most once," neatly preventing the problem.

A tour of `MainWindow.h`

A header typically declares a *class* (Chapter 7). Here is a shortened view of Idler's `MainWindow.h`:

```
#pragma once
#include <QMainWindow>
#include <QTimer>
```

```

class QPushButton; // "these types exist; details later"
class QLabel;

class MainWindow : public QMainWindow {
    Q_OBJECT
public:
    explicit MainWindow(QWidget* parent = nullptr);

private slots:
    void onStartStop();
    void onTick();

private:
    void stop();
    QPushButton* m_startStopBtn;
    QLabel*      m_statusLabel;
    bool         m_running = false;
};

```

Every function here is only *declared* — notice each ends in a semicolon with no body. The bodies live in `MainWindow.cpp`. The header is the *interface* (what the window can do); the `.cpp` is the *implementation* (how it does it). A reader who wants to know what a `MainWindow` offers reads the header; they only open the `.cpp` when they need the details.

Those lines like `class QPushButton;` are **forward declarations** — they promise the type exists so we can declare pointers to it, without pulling in its full definition. This keeps compilation fast.

A real linker error

Suppose you *declared* `performLeftClick()` in the header and called it, but forgot to *define* it in any `.cpp`. The code would compile fine — the compiler believes your promise — but the **linker** would fail with something like "undefined symbol: `performLeftClick`". This is the classic sign of a missing definition, and now you know exactly what it means: a promise with no body.

Note — The mental model: the header is a contract. The `.cpp` fulfills the contract. Other files trust the contract. If no file fulfills it, the linker is the one who notices, at the very end.

Chapter 6 — Teaching the Computer to Build: CMake

We have many source files, some Windows-only, some macOS-only, some needing extra libraries. Compiling them by hand with the right flags in the right order would be miserable. **CMake** automates it. You describe *what* you want built in a file called `CMakeLists.txt`, and CMake generates the actual build commands for your platform.

The two-step dance

Using CMake is always two commands. First **configure** (read the recipe, find your compiler and libraries, generate build files):

```
cmake -B build
```

Then **build** (actually compile and link):

```
cmake --build build
```

The `build` folder holds all the generated intermediate files, kept separate from your source. If it ever gets into a weird state, you can delete it and reconfigure — nothing of value is lost.

Reading Idler's CMakeLists.txt

Let us walk the important parts.

```
cmake_minimum_required(VERSION 3.16)
project(idler VERSION 1.0 LANGUAGES CXX OBJCXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

This names the project, declares which languages it uses (`CXX` is CMake's name for C++; `OBJCXX` is Objective-C++, needed for the macOS file), and demands the C++17 standard — the version of the language whose features we rely on.

```
find_package(Qt6 REQUIRED COMPONENTS Widgets)
```

This locates the Qt library on your machine. `REQUIRED` means "if you cannot find Qt, stop now with an error" — a clear failure is better than a confusing one later.

Choosing sources per platform

Here CMake shows its real value. We list the common source files, then add platform-specific ones only on the matching operating system:

```
set(SOURCES
    main.cpp
    MainWindow.cpp
    resources.qrc
)

if(WIN32)
    list(APPEND SOURCES clicker_windows.cpp assets/icon.rc)
elseif(APPLE)
    list(APPEND SOURCES clicker_mac.mm)
```

```
elseif(UNIX)
    list(APPEND SOURCES clicker_linux.cpp)
endif()
```

On Windows, `clicker_windows.cpp` joins the build; on macOS, `clicker_mac.mm`; on Linux, `clicker_linux.cpp`. The other two are never even shown to the compiler. One recipe, three platforms.

Producing the program and its libraries

```
qt_add_executable(idler WIN32 MACOSX_BUNDLE ${SOURCES})
target_link_libraries(idler PRIVATE Qt6::Widgets)
```

The first line says "build an executable named `idler` from these sources." The second says "link it against Qt's Widgets library" — connecting our calls to Qt's actual code, the linking step from Chapter 3.

Extra libraries, only where needed

Linux needs two X11 libraries to synthesize clicks. CMake adds them only on Linux:

```
if(UNIX AND NOT APPLE)
    find_package(X11 REQUIRED)
    target_link_libraries(idler PRIVATE ${X11_LIBRARIES} ${X11_Xtst_LIB})
endif()
```

Note — You do not need to memorize CMake syntax. You need to *recognize* what it is doing: finding tools and libraries, gathering the right source files for the current platform, and wiring them into a single program. It is the recipe; the compiler and linker are the cooks.

With the build system understood, we can finally dig into the star of the show: the class that *is* the window.

Chapter 7 — Thinking in Objects: Classes

So far our functions have floated freely. But Idler's window has *state* (is it running? how many clicks so far?) bundled together with *behavior* (start, stop, tick). The tool for bundling related state and behavior into one unit is the **class**.

What a class is

A class is a blueprint. It defines what data an object holds (its **member variables**) and what it can do (its **member functions**, also called **methods**). From the blueprint you create **objects** (also called **instances**).

Idler defines one central class, `MainWindow`. Its declaration lives in `MainWindow.h` and looks like this (trimmed):

```
class MainWindow : public QMainWindow {
    Q_OBJECT
public:
    explicit MainWindow(QWidget* parent = nullptr);

private slots:
    void onStartStop();
    void onTick();

private:
    void updateUI();
    void stop();

    QTimer m_clickTimer;
    bool    m_running    = false;
    int     m_clicksDone = 0;
};
```

Public and private: encapsulation

Notice the labels `public:` and `private:`. They control who may touch each member.

- **public** members are the class's outside interface — anyone can use them.
- **private** members are internal — only the class's own code may use them.

This is **encapsulation**, and it is one of the most important ideas in software. `m_running` and `m_clicksDone` are private: the outside world has no business poking at Idler's internal counters. They can only be changed through the window's own carefully written methods. This keeps the object always in a sensible state.

Note — The `m_` prefix on `m_running`, `m_clicksDone`, and friends is a naming convention for "member variable". When you are deep in a long method, the prefix instantly tells you "this belongs to the object and persists between calls," as opposed to a temporary local variable. Conventions like this are how large codebases stay readable.

The constructor

One method has a special role: the **constructor**, a function with the same name as the class. It runs automatically whenever an object is created, and its job is to put the new object into a valid initial state.

MainWindow's constructor is where the entire user interface is built — every button, box, and label is created and arranged there. Its declaration:

```
explicit MainWindow(QWidget* parent = nullptr);
```

And in MainWindow.cpp, its definition begins:

```
MainWindow::MainWindow(QWidget* parent) : QMainWindow(parent) {  
    setWindowTitle("Auto Clicker");  
    // ... create and lay out every control ...  
}
```

The :: scope operator

That `MainWindow::MainWindow` is worth pausing on. In the header we *declared* the methods inside the class. In the `.cpp` we *define* them outside the class body, so we must say which class each definition belongs to. `MainWindow::intervalSeconds` means "the `intervalSeconds` method that belongs to `MainWindow`." The `::` is the **scope resolution operator**; read it as "belongs to" or "of".

Why one big class?

Idler is small enough that a single `MainWindow` class holding the whole UI and its logic is appropriate. As programs grow, you split responsibilities across many classes. But the principle is the same at every scale: group the data and the behavior that belong together, expose a clean public interface, and hide the messy internals behind `private`.

Next we confront the trickiest topic for newcomers — pointers and memory — using Idler's own widgets as the examples.

Chapter 8 — Pointers, References, and Who Owns the Memory

You have already seen stars scattered through Idler's code: `QPushButton* m_startStopBtn`. That `*` marks a **pointer**, and pointers are the concept that most rewards careful attention. Let us build the idea slowly.

Two places to keep things: the stack and the heap

When a program runs it has two regions of memory:

- The **stack** — fast, automatic storage for local variables. When a function ends, its stack variables vanish automatically. Idler's `int intervalMs` inside a method lives here.
- The **heap** — a large pool for things that must outlive the function that created them. You explicitly ask for heap memory, and something must eventually give it back.

What a pointer is

A pointer is a variable whose value is the *address* of something else in memory — it points *at* another object rather than being that object. The type `QPushButton*` means "the address of a `QPushButton`." You create heap objects with the keyword `new`, which returns a pointer to the freshly made object:

```
m_startStopBtn = new QPushButton("Start");
```

This asks the heap for a new button and stores its address in `m_startStopBtn`. To use the object through a pointer, you use the arrow operator `->`:

```
m_startStopBtn->setText("Stop");
```

Read `->` as "reach through the pointer and use the member." Idler is full of this: `m_intervalSpin->value()`, `m_clickTimer.start(...)`, `m_statusLabel->setText(...)`.

The ownership question

Here is the hard part that C++ forces you to think about: if you asked the heap for memory with `new`, *who gives it back*? Memory that is never returned is a **leak**; memory returned twice, or used after being returned, is a **crash** or worse. In plain C++ you would pair every `new` with a matching `delete`.

But look closely at Idler — it calls `new` dozens of times and `delete` **zero** times. Why is that not a catastrophe?

Qt's parent-child ownership

The answer is a convention Qt builds on top of C++. When you create a Qt object, you can give it a **parent**. When a parent is destroyed, it automatically destroys all its children. Look at how the controls are created:

```
auto* central = new QWidget(this);
setCentralWidget(central);
```

```
auto* root = new QVBoxLayout(central);
```

The `this` passed to `new QWidget(this)` makes the `MainWindow` the widget's parent. That widget in turn becomes the parent of the layouts and controls placed inside it. The result is a *tree* of ownership. When the `MainWindow` is destroyed, it destroys its children, which destroy theirs, all the way down. Every new has an owner, so nothing leaks — without a single manual `delete`.

Note — This is a beautiful illustration of a general truth: raw C++ gives you total control and total responsibility, and good libraries layer *conventions* on top to make that responsibility manageable. Qt's parent-child system, C++'s own smart pointers (which you will meet later), and other patterns all exist to answer the one question: *who owns this, and when is it freed?*

`auto` — letting the compiler name the type

You will have noticed `auto* central = new QWidget(this);`. The keyword `auto` tells the compiler "figure out the type yourself from the right-hand side." Since `new QWidget(...)` obviously produces a `QWidget*`, writing it twice would be noise. `auto` keeps the code readable while staying fully statically typed — the type is still fixed and checked, you just did not have to spell it out.

`nullptr` — a pointer to nothing

A pointer must sometimes mean "points at nothing yet." That value is `nullptr`. You saw it as a default in `MainWindow(QWidget* parent = nullptr)`: "if no parent is given, use nothing." Calling `->` on a `nullptr` is a classic crash, so a pointer that might be null must be checked before use.

Pointers are the gateway to understanding how C++ programs really manage data. You do not need to master every subtlety today — you need to read Idler's `->` calls comfortably and understand *why* there are no `deletes`. That, you now do.

Chapter 9 — Building a GUI with Qt: Widgets, Layouts, the Event Loop

Everything until now has been groundwork. In this chapter the window finally appears on screen. The library that makes this possible is **Qt** (pronounced "cute"), a large, mature C++ toolkit for graphical applications.

The two Qt objects every app needs

Look again at `main.cpp`:

```
int main(int argc, char* argv[]) {
    QApplication app(argc, argv);
    app.setApplicationName("Auto Clicker");
    app.setWindowIcon(QIcon(":/assets/icons/icon_512.png"));

    MainWindow w;
    w.show();
    return app.exec();
}
```

- `QApplication app(...)` — every Qt program creates exactly one of these. It represents the application itself and manages global resources.
- `MainWindow w;` — this creates our window object (running its constructor, which builds all the controls). `w.show();` makes it visible.
- `return app.exec();` — this starts the **event loop**, the beating heart of any GUI program, explained below.

The event loop

A console program runs top to bottom and exits. A GUI program cannot work that way — it must sit and wait, possibly for minutes, until the user does something, then react, then wait again. `app.exec()` is an endless loop that does exactly this:

- Wait for an **event** (a mouse click, a key press, a timer firing).
- Find who should handle that event and call their code.
- Repeat.

Because `app.exec()` does not return until the window is closed, the `return` in `main` is only reached at the very end, as the program shuts down. Everything Idler does — responding to the Start button, ticking the timer — happens *inside* this loop, in response to events.

Widgets

A **widget** is any visible control: a button, a text box, a label, or the window itself. Idler's constructor creates many. A representative slice:

```
m_intervalSpin = new QDoubleSpinBox;
m_intervalSpin->setValue(1.0);

m_startStopBtn = new QPushButton("Start");
m_statusLabel = new QLabel("Stopped");
```

- `QDoubleSpinBox` — a numeric input with up/down arrows, for the interval value.
- `QPushButton` — the clickable Start/Stop button.
- `QLabel` — a line of read-only text, for the status message.
- `QComboBox` (used elsewhere) — the dropdown for choosing seconds/minutes/hours.

Each is created with `new` and, as Chapter 8 explained, owned by its parent so it is cleaned up automatically.

Layouts: arranging widgets without pixel math

You could position each widget by hand at exact coordinates, but then the window could not be resized and would look wrong on different screens. Instead Qt uses **layouts** — objects whose job is to arrange child widgets and adjust as the window changes. Idler stacks its sections vertically and places some controls side by side:

```
auto* root = new QVBoxLayout(central);    // vertical stack
auto* intervalRow = new QHBoxLayout;      // one horizontal row
intervalRow->addWidget(intervalLabel);
intervalRow->addWidget(m_intervalSpin);
intervalRow->addWidget(m_intervalUnit);
root->addLayout(intervalRow);
```

`QVBoxLayout` arranges its children top-to-bottom; `QHBoxLayout` left-to-right. By nesting a horizontal row inside the vertical stack, we get the label, the number box, and the unit dropdown sitting neatly on one line, with the next section below. The layout recalculates positions automatically whenever the window changes size.

Grouping related controls

Idler puts its three mode options inside a labelled box:

```
m_modeBox = new QGroupBox("Mode");
auto* modeLayout = new QVBoxLayout(m_modeBox);
m_radioInfinite = new QRadioButton("Infinite");
m_radioTimer = new QRadioButton("Stop after");
m_radioCount = new QRadioButton("Stop after");
m_radioInfinite->setChecked(true);
```

A `QRadioButton` is a mutually-exclusive choice — selecting one deselects the others, which is exactly right for picking a single mode. Grouping them in a `QGroupBox` both visually frames them and tells Qt they belong together.

We now have a window full of controls. But pressing the button does nothing yet. Connecting user actions to our code is the subject of the next chapter, and it is Qt's signature feature.

Chapter 10 — Making Things Happen: Signals and Slots

We have a Start button on screen. How does clicking it actually run our code? Qt's answer is an elegant mechanism called **signals and slots**, and it is worth understanding well because it shapes the entire architecture of a Qt program.

The idea

- A **signal** is an announcement that something happened. A `QPushButton` emits a `clicked` signal when pressed. A `QTimer` emits a `timeout` signal when its interval elapses.
- A **slot** is a function that can be run in response to a signal.
- `connect(...)` wires a signal to a slot: "when this happens, run that."

The beauty is *decoupling*. The button does not know or care what happens when it is clicked; it just announces "I was clicked." We decide separately what should follow.

Idler's connections

At the end of `MainWindow`'s constructor, all the wiring is done in one place:

```
connect(m_startStopBtn, &QPushButton::clicked, this, &MainWindow::onStartStop);
connect(&m_clickTimer, &QTimer::timeout, this, &MainWindow::onTick);
connect(m_radioTimer, &QRadioButton::toggled, this, &MainWindow::onModeChanged);
```

Read the first line as a sentence: "When `m_startStopBtn` emits `clicked`, call `onStartStop` on this object." The third argument, `this`, is the object whose slot should run — our window itself.

So the flow when the user clicks Start is:

- The button emits `clicked`.
- The event loop (Chapter 9) notices and looks up the connection.
- It calls `MainWindow::onStartStop()`.
- Our code runs.

The second connection is the one that drives the whole app: every time the timer fires, `onTick()` runs, which is where the actual mouse click happens.

The slots themselves

In the header, slots are declared under a special label:

```
private slots:
    void onStartStop();
    void onTick();
    void onModeChanged();
```

They are ordinary member functions; `slots` just marks them as connectable to signals. Here is the start/stop slot, now fully readable:

```
void MainWindow::onStartStop() {
    if (m_running) {
        stop();
    }
}
```

```

        return;
    }
    m_clicksDone = 0;
    m_elapsedSecs = 0.0;
    m_running = true;

    int intervalMs = qMax(50, static_cast<int>(intervalSeconds() * 1000.0));
    m_clickTimer.start(intervalMs);

    m_startStopBtn->setText("Stop");
    m_modeBox->setEnabled(false);
    updateUI();
}

```

One button, one slot, two behaviors: if already running, stop; otherwise reset the counters, start the timer, and update the interface. This is a common and clean pattern for a toggle control.

Q_OBJECT and the "MOC"

Remember the mysterious `Q_OBJECT` line at the top of the class? Signals and slots are not part of standard C++ — Qt adds them. To make them work, Qt runs a tool called the **Meta-Object Compiler** (MOC) over your headers *before* the real compiler. It sees `Q_OBJECT` and generates extra C++ code that implements the signal/slot machinery behind the scenes. CMake runs the MOC for you automatically (that is what `set(CMAKE_AUTOMOC ON)` in the build file switched on). You never see the generated code, but now you know why `Q_OBJECT` must be there: without it, the connections would not compile.

Note — Signals and slots are how Qt programs stay organized. Each widget minds its own business and simply announces events; a central place (`connect` calls) decides how those events flow into your logic. When you build your own Qt apps, thinking in terms of "what signals does this emit, and what should react to them?" will guide your whole design.

Chapter 11 — State and Time: The Timer and the Click Loop

Idler's entire purpose is to click *on a schedule*. This chapter ties the previous ones together by following what happens from pressing Start to the mouse actually clicking, and shows how the program keeps track of what it is doing — its **state**.

What is "state"?

State is simply the set of facts a program must remember between moments. Idler's state is small and lives in these member variables:

```
QTimer m_clickTimer;
bool   m_running    = false;
int    m_clicksDone = 0;
double m_elapsedSecs = 0.0;
```

"Are we running? How many clicks have we done? How much time has elapsed?" Every method reads and updates these facts. Managing state clearly is most of what programming *is*.

The QTimer

A `QTimer` emits its `timeout` signal repeatedly, at an interval you choose. We connected that signal to `onTick` in the previous chapter. Starting the timer, from `onStartStop`, is one line:

```
m_clickTimer.start(intervalMs);
```

From that moment, the event loop calls `onTick()` every `intervalMs` milliseconds, without any further effort from us. This is the event-driven mindset: we do not sit in a loop counting time ourselves; we ask the timer to notify us, and go back to waiting.

The tick

Here is the heart of the whole program:

```
void MainWindow::onTick() {
    performLeftClick();
    ++m_clicksDone;
    m_elapsedSecs += intervalSeconds();
    updateUI();

    if (m_radioCount->isChecked() && m_clicksDone >= m_clickCountSpin->value()) {
        stop();
        return;
    }
    if (m_radioTimer->isChecked() && m_elapsedSecs >= durationSeconds()) {
        stop();
    }
}
```

Trace it: perform the actual click (the function from `clicker.h!`), add one to the click counter, add the interval to the elapsed time, refresh the status text, then check the two stopping conditions. If we are in "N clicks" mode and have reached the target, stop. If we are in "duration" mode and have run long enough, stop. In "Infinite" mode neither `if` is ever true, so it clicks until the user stops it.

Two small pieces of syntax:

- `++m_clicksDone;` adds one to the variable. It is the idiomatic C++ way to increment a counter.
- `m_elapsedSecs += intervalSeconds();` is shorthand for `m_elapsedSecs = m_elapsedSecs + intervalSeconds();`.

Stopping cleanly

Whether the user presses Stop or a limit is reached, everything funnels through one method, so cleanup happens in exactly one place:

```
void MainWindow::stop() {
    m_clickTimer.stop();
    m_running = false;
    m_startStopBtn->setText("Start");
    m_modeBox->setEnabled(true);
    m_statusLabel->setText(QString("Stopped - %1 click(s) sent").arg(m_clicksDone));
}
```

It stops the timer (so `onTick` stops being called), flips the state back, restores the button's label and re-enables the controls that were locked while running. Having a single `stop()` that every path calls is a deliberate design choice: it guarantees the program can never end up half-stopped.

Note — The `QString(...).arg(m_clicksDone)` builds the status text by substituting the click count into the `%1` placeholder. `QString` is Qt's text type; you will use it constantly. Building messages this way, rather than gluing pieces together by hand, keeps text handling tidy and translatable.

Locking the controls while running

Notice `m_modeBox->setEnabled(false)` when starting and `true` when stopping. While Idler is clicking, it grays out the mode options so they cannot be changed mid-run — a small touch that prevents the program from getting into a confusing state. Thinking about *which states are valid* and forbidding the rest is a hallmark of careful programming.

You now understand the full life of a click. One question remains: how does `performLeftClick()` actually make the operating system click? That is where we go next.

Chapter 12 — One Codebase, Three Operating Systems

`performLeftClick()` has been our trusty black box: call it, and the mouse clicks. Now we open the box. And here we meet the most interesting engineering challenge in Idler: *the actual way to click the mouse is completely different on Windows, macOS, and Linux*. Yet the rest of the program should not have to care. How?

The strategy: one declaration, three definitions

Recall from Chapter 5 that `clicker.h` only *declares* the function:

```
void performLeftClick();
```

Every other file in the program includes this header and calls the function, knowing nothing about operating systems. The platform-specific knowledge is quarantined into three files, and CMake (Chapter 6) compiles only the correct one for the current platform. The rest of Idler is blissfully unaware.

This is a design pattern called an **abstraction layer**: a stable interface on top, swappable implementations underneath.

The preprocessor as a gatekeeper

Each platform file wraps its entire contents in a preprocessor guard so it compiles to *nothing* on the wrong platform. The Windows file begins:

```
#ifdef _WIN32
#include "clicker.h"
#include <windows.h>
// ... the Windows implementation ...
#endif
```

`#ifdef _WIN32` means "only include the following if the symbol `_WIN32` is defined," and the compiler defines `_WIN32` automatically when building on Windows. On macOS that symbol is absent, so the whole file collapses to emptiness. This is belt-and-suspenders alongside CMake's file selection, and it makes each file self-documenting about where it applies.

Windows: SendInput

```
#ifdef _WIN32
#include "clicker.h"
#include <windows.h>

void performLeftClick() {
    INPUT input[2] = {};
    input[0].type = INPUT_MOUSE;
    input[0].mi.dwFlags = MOUSEEVENTF_LEFTDOWN;
    input[1].type = INPUT_MOUSE;
    input[1].mi.dwFlags = MOUSEEVENTF_LEFTUP;
    SendInput(2, input, sizeof(INPUT));
}
#endif
```

Windows models a click as two events: the button going *down*, then *up*. We fill in a small array describing both and hand it to the operating system function `SendInput`. Notice a real-world truth surfacing: a "click" is not one thing, it is a press followed by a release.

macOS: CGEventPost

The macOS file has the extension `.mm` — that is **Objective-C++**, which lets us call Apple's system frameworks. The approach is the same shape, different names:

```
#ifdef __APPLE__
#include "clicker.h"
#include <ApplicationServices/ApplicationServices.h>

void performLeftClick() {
    CGEventRef down = CGEventCreateMouseEvent(
        nullptr, kCGEventLeftMouseDown, /*position*/, kCGMouseButtonLeft);
    CGEventRef up = CGEventCreateMouseEvent(
        nullptr, kCGEventLeftMouseUp, /*position*/, kCGMouseButtonLeft);
    CGEventPost(kCGHIDEventTap, down);
    CGEventPost(kCGHIDEventTap, up);
    CFRelease(down);
    CFRelease(up);
}
```

Again: create a "mouse down" event, create a "mouse up" event, post them to the system. And notice the `CFRelease` calls — Apple's C frameworks do *not* have Qt's automatic cleanup, so here we must manually release what we created. The same ownership question from Chapter 8, answered differently by a different library.

Linux: XTest

```
#ifdef __linux__
#include "clicker.h"
#include <X11/Xlib.h>
#include <X11/extensions/XTest.h>

void performLeftClick() {
    Display* display = XOpenDisplay(nullptr);
    if (!display) return;
    XTestFakeButtonEvent(display, Button1, True, CurrentTime);
    XTestFakeButtonEvent(display, Button1, False, CurrentTime);
    XFlush(display);
    XCloseDisplay(display);
}
#endif
```

On Linux the graphical system is (classically) X11, and its "XTest" extension can fake input events. We open a connection to the display, send button-1 down then up, flush the commands, and close the connection. Note the `if (!display) return;` — a defensive check, because opening the display can fail, and calling functions with a null display would crash.

Step back and admire the shape

Three files, three completely different APIs, three memory-management styles — all hidden behind a single line, `void performLeftClick()`. Any other part of Idler, and any future feature, uses the mouse without ever thinking about operating systems. When someone eventually ports Idler to a new platform, they write one new file and change nothing else.

Note — This is perhaps the most valuable lesson in the entire book. Complexity is not eliminated — clicking really is different on every OS — but it is *contained*. Good software is not software with no hard parts; it is software where the hard parts are sealed behind clean, simple interfaces. Learn to find the `performLeftClick()` in every problem you face.

Chapter 13 — Shipping It: Building, Bundling, Distributing

A program that only runs on your own machine is a hobby. Turning it into something a stranger can download and run is what makes it *software*. This chapter surveys how Idler goes from source code to a distributable app on each platform. You do not need to master every command; aim to understand the shape of the problem.

Debug builds versus release builds

While developing, you build in *debug* mode: slower, but full of checks and information for finding bugs. To give to users, you build in *release* mode: optimized for speed and size. With CMake that is one flag:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

The bundling problem

Idler depends on Qt. On your machine Qt is installed, so the program finds it. On a stranger's machine, Qt is *not* installed — so if you just hand them the bare executable, it will refuse to start, complaining about missing libraries. The solution is to **bundle** the needed Qt libraries alongside your program. Each platform has a tool for this.

Windows

The tool `windeployqt` inspects the executable and copies every Qt DLL (dynamic library) it needs into the same folder. Idler's build script automates it, producing a self-contained folder the user can unzip and run — no installation required. This is a "portable" app.

macOS

macOS apps are really *folders* in disguise, called **bundles** (ending in `.app`), containing the executable, its libraries, and its icon. The tool `macdeployqt` copies Qt's frameworks inside the bundle. Idler's script then wraps the `.app` into a `.dmg` disk image — the familiar "drag the icon to Applications" installer.

Linux

Linux users typically build from source (as in Chapter 2) or install through a package. Idler's CMake file includes rules to install the program and its icons into the standard system locations.

Icons and identity

An app needs a face. Idler ships one icon drawn once and converted into each platform's required format: `.ico` for Windows, `.icns` for macOS, and a set of PNG sizes for Linux. The build files wire the right format into each platform's build so the app shows its icon in the taskbar, Dock, and window title.

A word on trust: code signing

Modern operating systems are wary of programs from unknown sources. macOS in particular will warn users that an unsigned app "cannot be verified." Professional apps are **code signed** with a certificate that proves who made them (and on macOS, **notarized** by Apple). This costs money and paperwork, so a personal project like Idler is usually distributed unsigned, with instructions for the user to approve it manually the first time. Knowing this vocabulary — *signing*, *notarization*, *Gatekeeper* — is part of understanding real-world software distribution.

Note — The lesson of this chapter is that writing the code is only half the job. Getting it to run reliably on a machine you have never seen, made by a person who has never met you, is its own discipline — full of libraries, formats, and trust. Every real product crosses this bridge.

You have now followed Idler from a blank folder to a signed-and-shipped application, understanding every layer. In the final chapter, we choose your next climb.

Chapter 14 — Next Lesson: Build a Focus Timer

You have read and understood a complete, real, cross-platform C++ application. The best way to cement that knowledge is to build something new that *reuses* what you know while stretching you into fresh territory. This chapter is your next project brief.

The project: "Focus" — a Pomodoro timer

The **Pomodoro Technique** is a simple productivity method: work for 25 minutes, then take a 5-minute break, and repeat. Your next app is a desktop timer that automates this cycle. It is the perfect follow-up to Idler because the skeleton is familiar — a Qt window, a `QTimer`, a Start/Stop button, state to track — while every genuinely new idea it introduces is one you are now ready for.

What you already know how to do

Because you built Idler, the following will feel routine:

- Create the project with CMake and Qt.
- Lay out a window with labels, buttons, and spin boxes.
- Drive the countdown with a `QTimer` and a tick slot.
- Track state (working vs. on break, time remaining) in member variables.
- Funnel start/stop/reset through clean, single-purpose methods.

The new concepts it will teach you

Each of these is a natural next step, and together they round out a beginner into someone who can build genuinely useful tools:

- **Saving settings to disk.** A Pomodoro timer should remember your preferred work and break lengths between runs. Qt's `QSettings` class reads and writes small configuration values to the right place on every OS. This is your introduction to *persistence* — programs that remember.
- **Desktop notifications.** When a session ends, the app should notify you even if it is in the background. Qt's `QSystemTrayIcon` can show system notifications. This teaches you to interact with the OS's notification center — a new kind of platform integration, echoing Idler's `performLeftClick()`.
- **A system tray icon.** A timer belongs in the menu bar / system tray, not hogging a window. Putting an icon there with a small menu (Start, Pause, Quit) teaches you a second style of user interface beyond the plain window.
- **Formatting time for humans.** Turning 90 remaining seconds into a clean "01:30" display teaches you string formatting and integer math (division and modulo) in a concrete, visible way.
- **A richer state machine.** Idler was running-or-stopped. A Pomodoro has more states: working, short break, long break, paused. Modeling these cleanly — perhaps with an `enum` (a named set of states, a tool you have not met yet) — is excellent practice in the state-management thinking that Chapter 11 introduced.

Suggested milestones

Build it in stages, running the program after each stage. This habit — always having a working program, growing it in small verified steps — is itself one of the most important professional skills.

- **Milestone 1.** A window with a fixed 25:00 countdown, a Start button, and a label that updates every second. (Pure Idler knowledge.)
- **Milestone 2.** Add Pause and Reset. Introduce an `enum` for the state. Format the remaining time as `MM:SS`.
- **Milestone 3.** After a work session ends, automatically start a 5-minute break, then loop back. Now you have the full Pomodoro cycle.
- **Milestone 4.** Let the user configure work/break lengths, and remember them with `QSettings` so they persist after closing the app.
- **Milestone 5.** Show a desktop notification when each session ends, and add a system tray icon with a menu.
- **Milestone 6.** Package it for your OS using what Chapter 13 taught, icon and all.

Stretch goals, once it works

- Play a gentle sound at the end of each session (Qt Multimedia).
- Count how many Pomodoros you complete each day and save the tally.
- A small settings dialog window (your first *second* window — teaches multi-window apps).

A closing word from your instructor

You began this book unable to read a line of C++. You end it able to follow a real application across the language, its build system, a major GUI framework, memory ownership, and three operating systems' native APIs — and you have a concrete, well-scoped next project waiting.

The secret the whole way through has been the same: complexity is tamed by dividing it into small, well-named pieces with clean boundaries — a header that promises, a `.cpp` that delivers, a `performLeftClick()` that hides an entire operating system behind one honest line. Carry that instinct into the Focus timer, and into everything you build after it.

Now close the book, open your editor, and build.